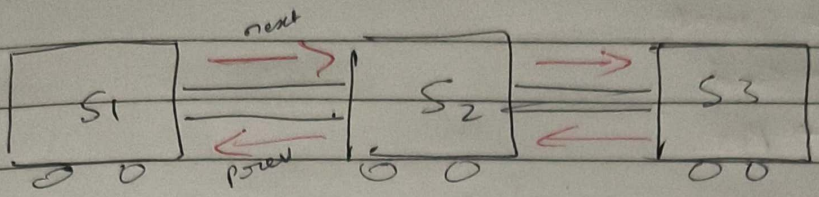
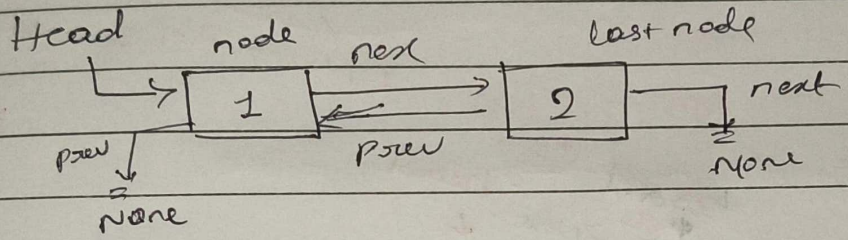
16/8/25
Day 33

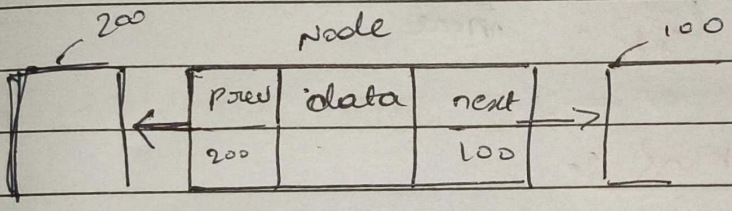
Date :
Page No. :

# 13 Doubly linked list - Theory :



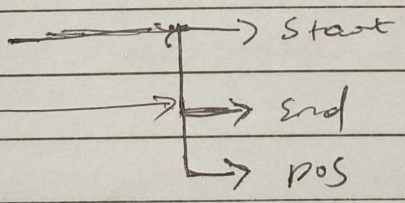

## DLL

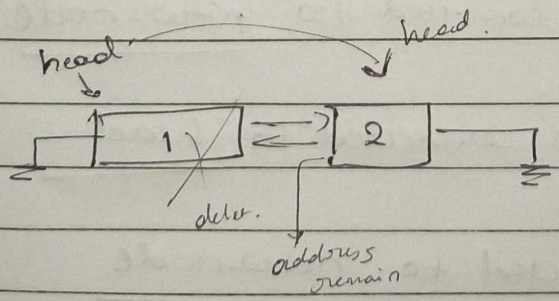



| Undo / Redo |
| --- |
| ctrl + z or ctrl + u |

-> Insert
-> Delete
-> Search
-> Print




then None it.

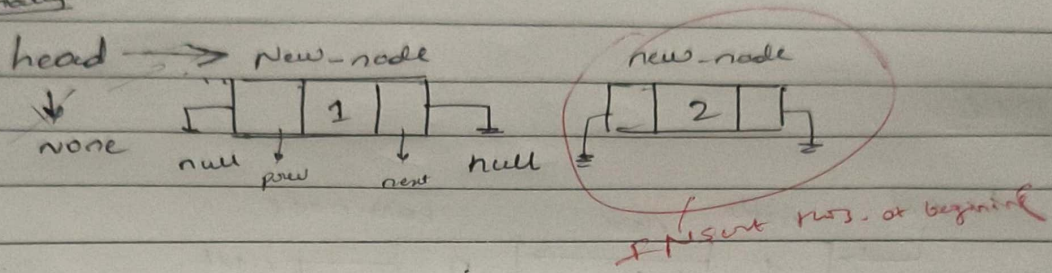
17 Insert

↳ Start

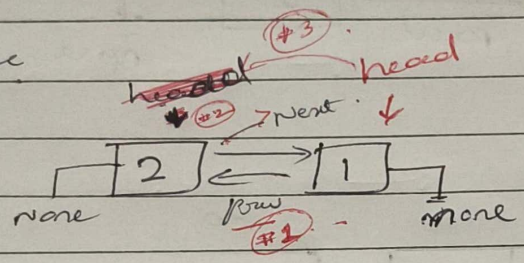
↳ Snd

↳ Pos

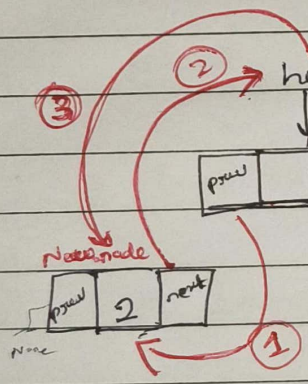
Initially




Look like




* Insert at beginning :-




* 1st write always right to left ←

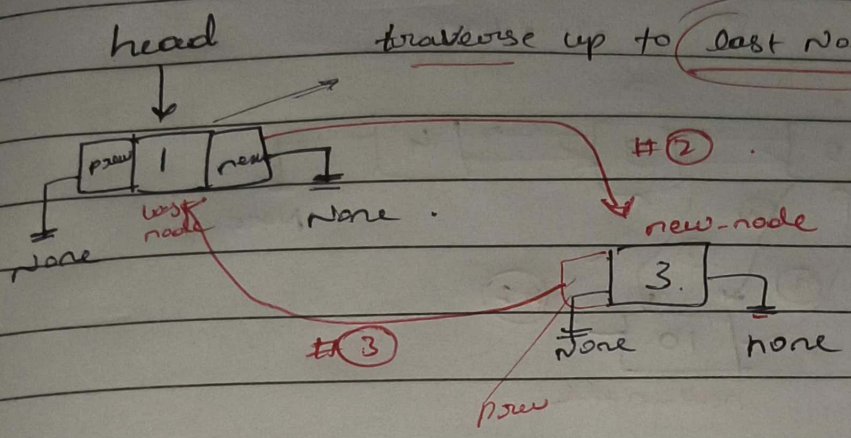
① 1st node prev is connected to New-node

② new-node next is connected to head.

③ head is connected to new-node

Date :...../...../.....
Page No. :.....

# Insert at End




#4 list Empty (direct point the head to into new-node)

(1) Check the last node means traverse up to last node.

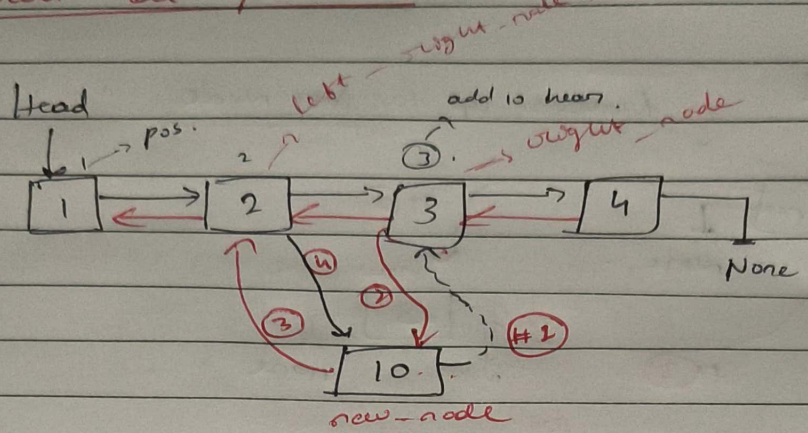
(2) last-node .next → new-node

(3) new-node . prev → last-node

# (4) list is Empty

head → new-node.

## Insert at position :-




① new_node.next → pos of node. (3) (right_node)

② pos of node.prev → new_node. (3) (right_node)

③ new_node.prev → pos of back of pos node (2) (left_node)

④ back of pose node.next → new_node (2) (left_node)

order not mattering

new_node.next = right_node

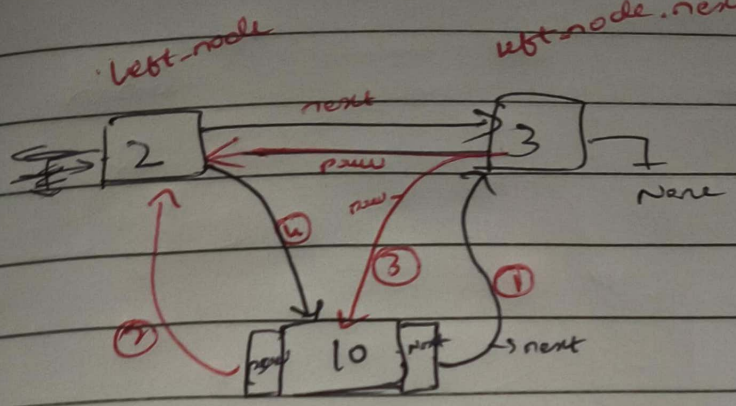
right_node.prev = new_node

new_node.prev = left_node

left_node.next = new_node

Date :...../...../.....
Page No. :.....

Q2) for one variable :- (order matters)




order matters

Code :-

① new_node.next = left_node.next

② new_node.prev = left_node

③ left_node.next.prev = new_node

④ left_node.next = new_node